Task 1 — Web App QA & Debug Report

Tester: Sahil Pawar | **App under test:** Conduit (RealWorld Angular frontend demo) | **URL:** https://demo.realworld.show | **API:** https://api.realworld.show | **Date:** 2026-09-29

Note on app selection

The assignment's suggested demo URL (demo.realworld.io) is dead — it returns an S3 NoSuchBucket 404, because the gothinkster/realworld repo has moved to realworld-apps/realworld and the live demo moved to demo.realworld.show. I used the current live demo instead.

Methodology

I tested at two levels. First, the normal user journey through the UI: sign-up, login, create/edit/delete an article, comment, favorite, logout — deliberately feeding bad input (empty forms, invalid emails, 1-character passwords, non-existent URLs, an XSS payload) since real bugs live in edge cases. Second, once I noticed the frontend and backend disagreed on a few things, I went one level down and called the REST API directly (curl) to see what the server actually enforces versus what the UI merely hides — this is where the more serious findings came from. For anything destructive (ownership/authorization testing), I only ever used two throwaway test accounts I created myself, never the shared seed data (johndoe, janesmith, etc.) that other candidates might also be testing against, and I cleaned up every test article/comment afterward.

Bug Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Sign-up allows invalid emails and 1-character passwords — and the server itself is inconsistent about it	1. POST /api/users with password "1" → 201 Created. 2. Log in as that user, then PUT /api/user with the same "1" password → 422 “password must be a string between 8 and 128 chars”.	**Expected:** Same password policy enforced everywhere. **Actual:** Registration accepts a 1-char password and any string as “email”; the settings-update endpoint rejects the identical password. Two code paths, two different rules.	Critical	The registration handler and the profile-update handler were clearly written independently and never shared a validation rule — classic symptom of AI-scaffolded endpoints built one at a time without a shared schema.
2	“Delete Article” is instant and irreversible — no confirmation	1. Log in, open any of your own articles. 2. Click “Delete Article”.	**Expected:** A confirmation prompt before an irreversible action. **Actual:** Article deleted immediately on one click, no dialog, no undo.	High	Delete button's click handler calls the delete API directly — no confirmation modal was built in.
3	Non-existent article URL renders a completely blank page	1. While logged in, go directly to a made-up slug, e.g. /article/does-not-exist-12345.	**Expected:** A “404 — Article not found” message or redirect. **Actual:** Page is entirely blank below the navbar. Browser console shows the API correctly returned HTTP 404, but the frontend never displays anything for it.	High	The article component's data-fetch has no error branch for a failed lookup — the happy path was built and tested, the failure path never was.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
4	API's CORS policy trusts any website on the internet	Sent an OPTIONS preflight to /api/users/login with Origin: https://evil-attacker-site.example → response included access-control-allow-origin: https://evil-attacker-site.example and access-control-allow-credentials: true.	**Expected:** Access-Control-Allow-Origin restricted to the app's own domain(s). **Actual:** The API reflects back whatever Origin header it receives, i.e. it trusts every website equally.	High	CORS middleware configured with a wildcard/reflect-any-origin policy instead of an explicit allow-list — a common copy-paste default that's rarely revisited.
5	No security response headers anywhere	curl -I on both https://demo.realworld.show and https://api.realworld.show.	**Expected:** Standard hardening headers present (CSP, X-Frame-Options, Strict-Transport-Security, X-Content-Type-Options). **Actual:** None of them are set on either the frontend or the API.	Medium	No reverse-proxy/security-header middleware was configured — the app is running with framework/server defaults only.
6	Login/session token stored in localStorage, not an HttpOnly cookie	In browser DevTools console: localStorage.getItem('jwtToken').	**Expected:** Session token kept out of reach of page JavaScript (HttpOnly cookie), so a future XSS bug can't steal it. **Actual:** The token sits in localStorage in plain text, readable by any script running on the page.	Medium	Simplest way to implement token-based auth in a client-side SPA tutorial app — fine for a demo, a real risk multiplier if any XSS hole is ever introduced.

What I checked and found to be done correctly

To be clear this wasn't a one-sided hunt — I also verified a few things the app gets right, since knowing what's *not* broken matters too: (1) **Authorization is enforced server-side, not just hidden in the UI.** I created two throwaway accounts and confirmed a non-owner gets a real 403 trying to edit another user's article via the API directly (not just a hidden button), and a 404 trying to delete another user's comment. (2) **No user enumeration on login** — a non-existent email and a wrong password both return the identical generic error. (3) **XSS payloads in article titles/bodies are safely escaped** — an injected <script> and an onerror handler both rendered as inert text, not executable code.

Root-Cause Analysis — Bug #1 (Inconsistent sign-up validation / weak password acceptance)

I picked this one for the deep-dive because I could prove it two different ways, which makes it the most defensible finding rather than the most dramatic-sounding one. First, at the UI level: the Register page's email field is a plain type="text" input, while the near-identical Settings page correctly uses type="email" — so the browser's native email-format check never even fires on sign-up. Second, and more importantly, at the API level, which is the layer that actually matters for security: I called POST /api/users directly with a password of a single character and it returned 201 Created — account made, no complaint. I then logged in as that account and called PUT /api/user (the settings-update endpoint) with that exact same password, and it was rejected with a 422 and the message “password must be a string between 8 and 128 chars.” That's the same backend, two different endpoints, two different rules for the same field. The most likely explanation is that these two endpoints were built or AI-generated separately — the update endpoint got a proper validator at some point (maybe copied from a tutorial or added when a reviewer noticed weak passwords), while the original registration endpoint was left with the validator it shipped with, which does nothing. Neither endpoint validates email format at all, at any layer, which I also confirmed directly via the API. In a real product this means anyone can create accounts with unreachable emails (breaking password-reset/notification flows) and single-character passwords (trivial to guess or brute force), and the one endpoint that does check password strength gives a false sense that the app is protected when it plainly isn't at the front door. The fix is to write the validation rules once, in one shared place (a schema or a reusable validator function), and have both the registration and update handlers call it — instead of each handler inventing its own rules — plus enforce it server-side regardless of what the client sends, since client-side checks (like input type="email") are a UX nicety, not a security boundary.

Evidence for the API-level findings (raw requests)

```
POST /api/users {"user":{"username":"qa_weakpw_check","email":"qa_weakpw_check@example.com","password":"1"}}
→ 201 Created

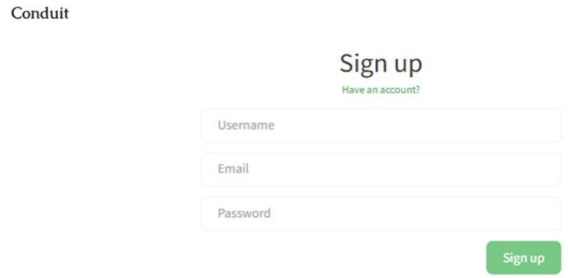
PUT /api/user (same account, same password "1")
→ 422 {"errors":{"body":["password must be a string between 8 and 128 chars"]}}

OPTIONS /api/users/login Origin: https://evil-attacker-site.example
→ 200 access-control-allow-origin: https://evil-attacker-site.example
access-control-allow-credentials: true

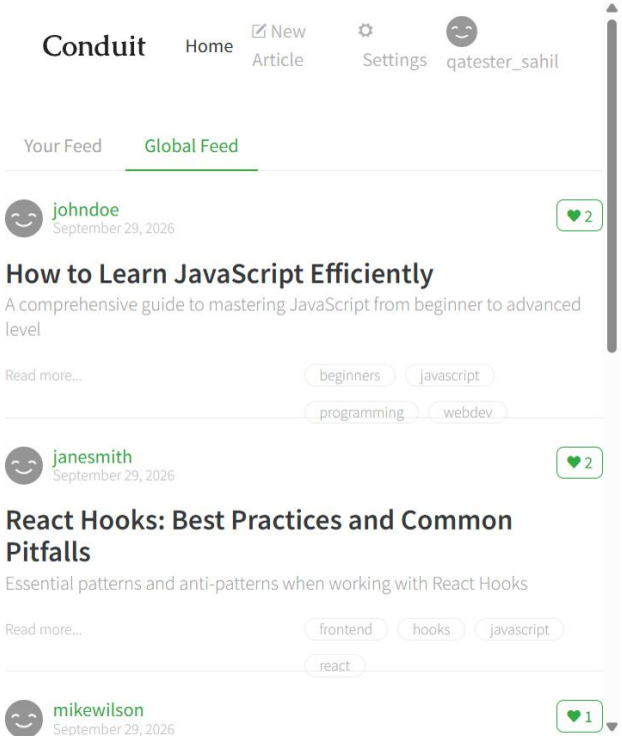
curl -I https://api.realworld.show/api/articles?limit=1
→ no Content-Security-Policy / X-Frame-Options / Strict-Transport-Security / X-Content-Type-Options headers present
```

Screenshots

Sign-up form (Bug #1 context) — no client-side email/password format restrictions in the UI



Logged in immediately after registering with an invalid email + 1-char password



Blank page for a non-existent article slug (Bug #3)

